

Lid-Driven Flow with Lattice Boltzmann Method

ME 7751 Homework 6

Toby Viet Nguyen

April 13, 2026

Scripts Included

`solver.py`, `finite_differences.py`, `plotting.py`, `plot_csv.py`, `part2a.sh`, `part2b.sh`, `part3a.sh`, `part3b.sh`

Usage Instructions

All Bash scripts call the functionality to `solver.py` and `plot_csv.py`. `solver.py` imports functions from `finite_differences.py` and `plotting.py`, which include code reused from previous assignments. Run any script to regenerate all the images used for its corresponding part. Run `python solver.py -h` to see the available command-line arguments.

1. Description of Lattice Boltzmann Method

The Lattice Boltzmann method can be used to simulate fluid dynamics by modeling the fluid as a collection of particles moving on a discrete lattice. The method solves the Boltzmann equation which describes the evolution of the particle distribution function $f(t, \vec{x}, \vec{p})$ over time. The Boltzmann equation is given by

$$\frac{Df}{Dt} = \frac{\partial f}{\partial t} + \vec{v} \cdot \frac{\partial f}{\partial \vec{x}} + \vec{F} \cdot \frac{\partial f}{\partial \vec{p}} = \Omega(f) \quad (1)$$

where $\Omega(f)$ is the collision operator that models the interactions between particles. In the Lattice Boltzmann method, the particle distribution function is discretized in both space and velocity, leading to a set of discrete velocities $\mathbf{c}_i$ and corresponding distribution functions $f_i(\mathbf{x}, t)$. The evolution of these distribution functions is governed by the lattice Boltzmann equation, which can be solved iteratively to simulate the fluid flow. The governing equation for the Lattice Boltzmann method is given by

$$f_i(\mathbf{x} + \mathbf{c}_i \Delta t, t + \Delta t) - f_i(\mathbf{x}, t) = \Omega(f_i) \quad (2)$$

This implementation uses a D2Q9 lattice, meaning the particles can move in 9 discrete directions (including rest) on a 2D lattice. An example of a D2Q9 lattice is shown in **Figure 1**. In lattice Boltzmann units, the lattice spacing Δx and time step Δt are both set to 1, and the particle velocities are defined as follows

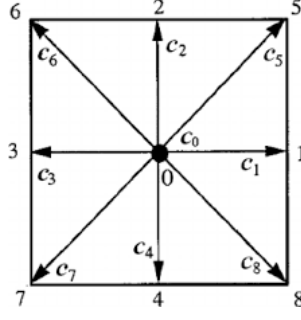


Figure 1: Schematic of the D2Q9 lattice. A particle located at the center of the lattice (black dot) can move in 9 discrete directions: rest (0), right (1), up-right (2), up (3), up-left (4), left (5), down-left (6), down (7), and down-right (8). The arrows indicate the possible directions of particle movement.

$$\mathbf{c}_0 = (0, 0) \quad (\text{rest}) \quad (3)$$

$$\mathbf{c}_1 = (1, 0) \quad (\text{right}) \quad \mathbf{c}_2 = (1, 1) \quad (\text{up-right}) \quad (4)$$

$$\mathbf{c}_3 = (0, 1) \quad (\text{up}) \quad \mathbf{c}_4 = (-1, 1) \quad (\text{up-left}) \quad (5)$$

$$\mathbf{c}_5 = (-1, 0) \quad (\text{left}) \quad \mathbf{c}_6 = (-1, -1) \quad (\text{down-left}) \quad (6)$$

$$\mathbf{c}_7 = (0, -1) \quad (\text{down}) \quad \mathbf{c}_8 = (1, -1) \quad (\text{down-right}) \quad (7)$$

This implementation also uses the Bhatnagar-Gross-Krook (BGK) collision operator, which relaxes the distribution function towards an equilibrium distribution f_i^{eq} at a rate determined by the relaxation time τ . The BGK collision operator is given by

$$\Omega(f_i) = -\frac{1}{\tau}(f_i - f_i^{eq}) \quad (8)$$

The equilibrium distribution function f_i^{eq} is typically defined based on the local density ρ and velocity $\mathbf{u}$ of the fluid, and can be computed using the following formula

$$f_i^{eq} = a_i \rho \left(1 + \frac{\mathbf{c}_i \cdot \mathbf{u}}{c_s^2} + \frac{(\mathbf{c}_i \cdot \mathbf{u})^2}{2c_s^4} - \frac{\mathbf{u} \cdot \mathbf{u}}{2c_s^2} \right) \quad (9)$$

where a_i are the weights associated with each discrete velocity direction and $c_s = \frac{1}{\sqrt{3}}$ is the speed of sound in lattice Boltzmann units. The weights for the D2Q9 lattice are given by

$$a_0 = \frac{4}{9} \quad (\text{rest}) \quad (10)$$

$$a_1 = a_3 = a_5 = a_7 = \frac{1}{9} \quad (\text{cardinal directions}) \quad (11)$$

$$a_2 = a_4 = a_6 = a_8 = \frac{1}{36} \quad (\text{diagonal directions}) \quad (12)$$

The macroscopic density ρ and velocity $\mathbf{u}$ can be computed from the distribution functions as follows

$$\rho(\mathbf{x}, t) = \sum_i f_i(\mathbf{x}, t) \quad (13)$$

$$\mathbf{u}(\mathbf{x}, t) = \frac{1}{\rho} \sum_i f_i(\mathbf{x}, t) \mathbf{c}_i \quad (14)$$

To enforce the no-slip boundary condition at the walls, we use the half-way bounce-back scheme, which reflects the distribution functions back into the domain when they encounter a boundary. For example, if a particle moving to the right (direction 1) hits the right wall, it will be reflected back in the opposite direction (direction 5). This ensures that the velocity at the wall is zero, satisfying the no-slip condition. For the moving lid at the top boundary, we add a momentum correction term to the distribution functions to account for the velocity of the lid.

$$f_{i'} = f_i - 2a_i\rho\frac{\mathbf{c}_i \cdot \mathbf{u}_{\text{lid}}}{c_s^2} \quad \text{for } i \in \{2, 3, 4\} \quad (15)$$

Here, i' is the direction opposite to i (e.g., if $i = 2$ (up-right), then $i' = 6$ (down-left)). This correction ensures that the momentum of the fluid near the moving lid is properly accounted for, allowing us to simulate the lid-driven flow accurately.

Putting it all together, the Lattice Boltzmann method iteratively updates the distribution functions at each time step by first performing the collision step using the BGK operator, then streaming the distribution functions to their new positions based on their discrete velocities, and finally applying the appropriate boundary conditions. This process continues until the solution converges to a steady state, which can be determined by checking if the max change in the velocity components u, v between successive time steps is below 10^{-12} .

2. Steady State Solution for $Re = 100$ and $Re = 400$

To compute the steady state solution for $Re = 100$ and $Re = 400$, we run the solver with $N = 128, U = 0.1$ until the velocity field converges to a steady state. The resulting steady state streamlines, divergence, and centerline velocities are plotted in **Figure 2**.

As expected, the streamlines show a single primary vortex near the center of the domain and multiple smaller vortices in the corners of the domain, matching what we see in real lid-driven flow. From roughly comparing the location of the center vortex and the size of the corner vortices with those in Ghia et al. (1982) (see **Figure 3**), the numerical solution matches the literature solution reasonably well for both $Re = 100, U = 0.1$ and $Re = 400, U = 0.5$. The divergence plots show that the velocity field is approximately divergence-free, with values close to zero throughout the domain. Sharp peaks in divergence are present at the top corners, which become negligible with larger N . The x-velocity and y-velocity profiles along the vertical and horizontal centerlines, respectively, roughly match the literature values.

3. Qualitative Spatial Convergence Rate

To analyze the spatial convergence rate of the solver, we run the solver for $Re = 100, U = 0.1$ and $Re = 400, U = 0.5$ with different grid resolutions ($N = 16, 32, 64, 128$ for $Re = 100$ and $N = 32, 64, 128$ for $Re = 400$). The resulting steady state centerline velocities are plotted in **Figure 4**. Through visual inspection, we can see that the centerline velocities for $N = 16, 32, 64$ approach that for $N = 128$ as N increases, indicating that the solution is converging to a more accurate representation of the flow field.

4. Quantitative Spatial Convergence Rate

To quantitatively analyze the spatial convergence rate of the solver, we compute the mean absolute error (MAE) between the numerical solution using $N = 16, 32, 64$ and the numerical solution using $N = 128$. Linear interpolation is used to sample the numerical solutions for $N = 16, 32, 64$ at 128 evenly distributed points, allowing the MAE to be calculated using 128 grid points. The resulting MAE values for $Re = 100$ and are plotted against the grid spacing h in **Figure 5**.

It is clear to see that the spatial convergence rate doesn't match the theoretical second-order convergence rate of the finite difference scheme. Instead, the slope of the dotted line in **Figure 5** is approximately 1.4, indicating a convergence rate of approximately 1.4. This discrepancy could be

due to several factors, including the presence of numerical errors from the boundary conditions and the fact that we are comparing against a numerical solution rather than an exact analytical solution. Additionally, the convergence rate may be affected by the choice of relaxation time τ in the Lattice Boltzmann method, which can introduce additional sources of error if not chosen appropriately.

5. Discussion of Method Stability and Stability Criteria

In order to maintain stability of the Lattice Boltzmann method with a given Re and N , it is important to choose the lid velocity U appropriately. For stability, the relaxation time τ must be ideally greater than 0.6 and the Mach number must be less than 1. In lattice Boltzmann units, we can express the relaxation time τ and Ma in terms of U as follows

$$\tau = \frac{\nu}{c_s^2} + \frac{1}{2} = \frac{UL}{Rec_s^2} + \frac{1}{2} > 0.6 \quad (16)$$

$$Ma = \frac{U}{c_s} < 1 \quad (17)$$

Here, ν is the kinematic viscosity of the fluid, which can be expressed in terms of U , $L = N - 1$, and Re as $\nu = \frac{UL}{Re}$. Rearranging the inequalities for τ and Ma , we can derive the following stability criteria for U

$$\frac{0.1Rec_s^2}{L} < U < c_s = \frac{1}{\sqrt{3}} \approx 0.577 \quad (18)$$

As N decreases, the lower bound for U increases, which can make it more difficult to maintain stability while also ensuring that the flow remains in the laminar regime. This became an issue when trying to run the solver for $Re = 400$ with $N = 16$, as the lower bound for U was approximately 0.889, which is greater than c_s . This suggested that for these conditions, there is no U which would ensure stability.

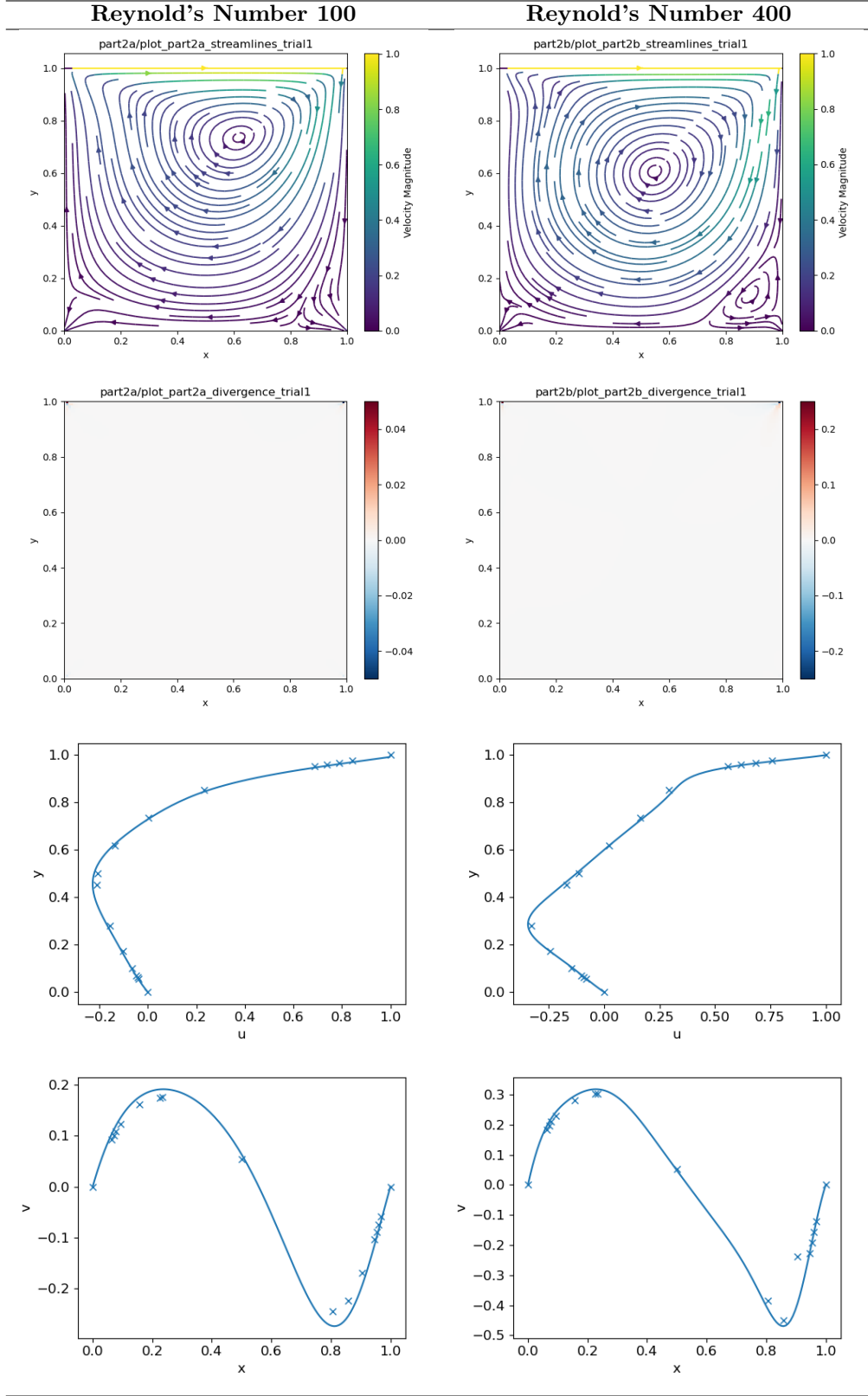


Figure 2: Steady state streamlines (first row), divergence (second row), vertical centerline x-velocities (third row), and horizontal centerline y-velocities (fourth row) for $Re = 100, U = 0.1$ and $Re = 400, U = 0.5$ with $N = 128$. Literature values for the centerline velocities are marked with X's for comparison.

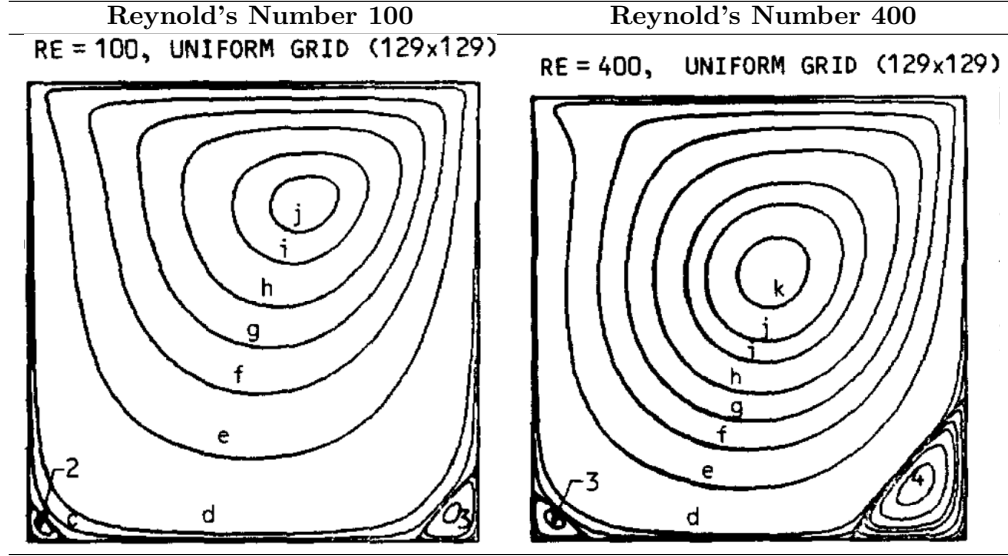


Figure 3: Literature streamline plots for $Re = 100$ and $Re = 400$ from Ghia et al. (1982) for comparison with our numerical solution.

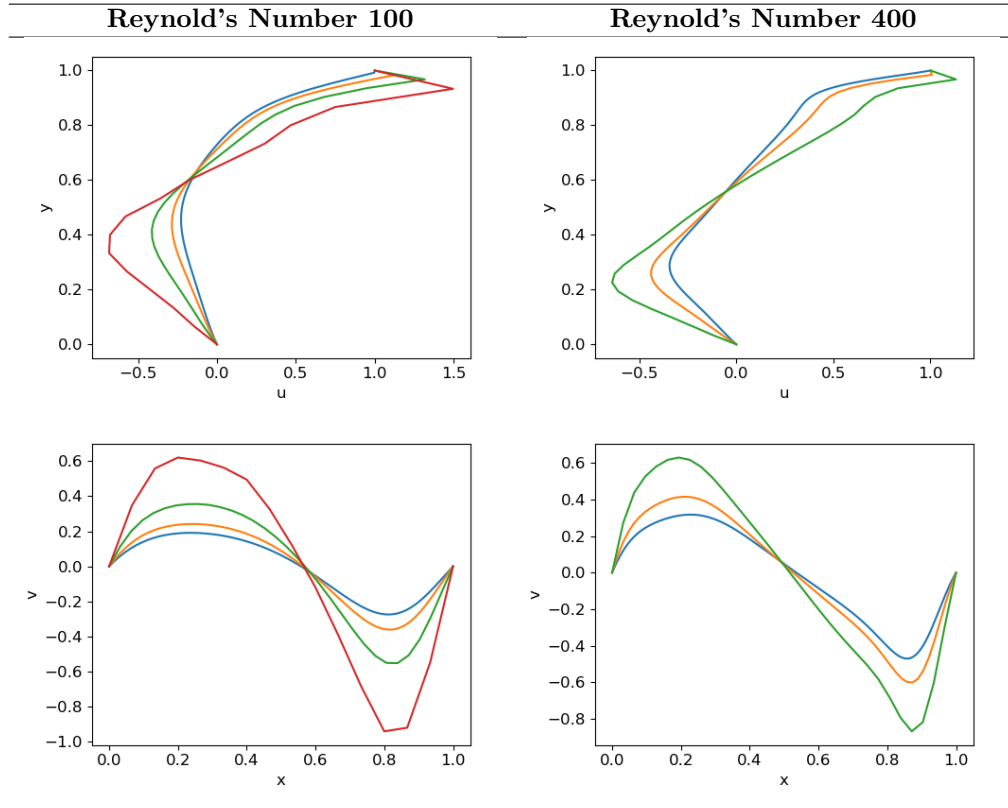


Figure 4: Centerline x-velocity profiles for $Re = 100, U = 0.1$ and $Re = 400, U = 0.5$ with different grid resolutions ($N = 16$ (red), 32 (green), 64 (orange), 128 (blue)).

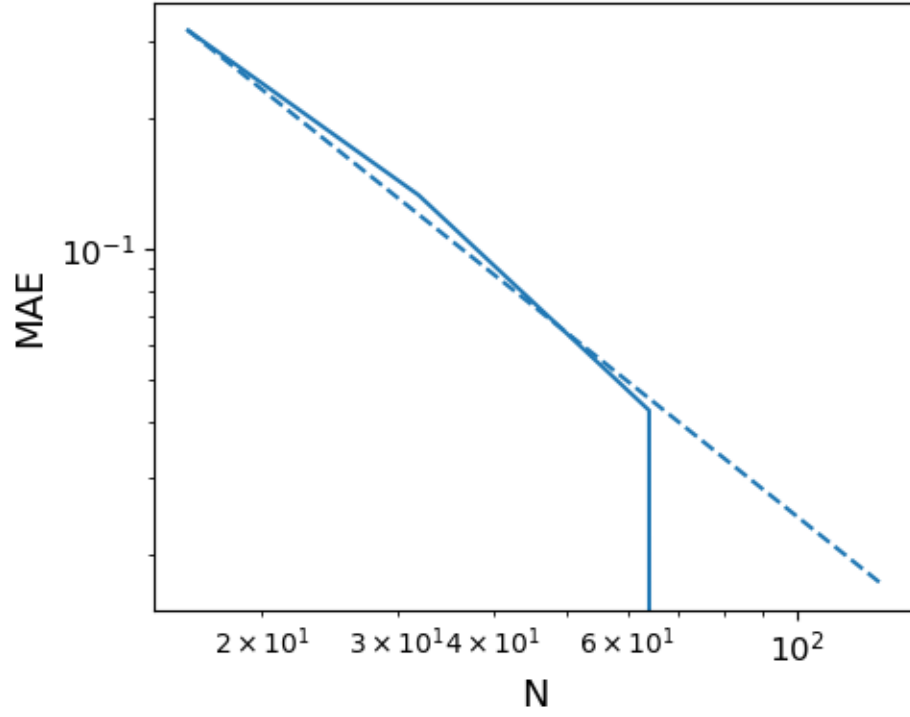


Figure 5: Mean absolute error (MAE) between the numerical solutions for $N = 16, 32, 64$ and the numerical solution for $N = 128$, using $Re = 100, U = 0.1$. The plot is on a log-log scale to better visualize the convergence rate. The slope of the dotted line is 1.4, indicating a convergence rate of approximately 1.4.